

PROJECT ARCHITECTURE PACKAGE

AI/ML-Based Campus Maintenance Reporting System

INNOVEXA — Capstone Design Project

Team: ISLAM MD KAYES • ZEHAD AMZAD BIN MIR • PRANTO MD TAIF RAHMAN
AI & Computer Engineering Department | 2026 2nd Semester

UML Use Case • Activity • Sequence • Mockup/Wireframe • ER Diagram

1. UML — Use Case Diagram

Actors and use cases follow the submitted project workflow: student reporting, AI/ML analysis, maintenance dispatch, status tracking, escalation, and administrator analytics.

Actor	Use Cases
Student	Login; submit facility report; upload photo; enter location; track status; receive notifications; provide feedback
Maintenance Staff	View assigned tasks; accept task; update status; add maintenance notes; mark resolved
Administrator	Monitor reports; manage departments; monitor workload; view analytics; review escalated issues; review recurring problems
AI/ML Engine	Classify issue; detect duplicate; detect predefined hazards; assess severity/safety; predict priority; recommend department
Notification/Escalation Service	Send status notification; monitor response target; send reminder; notify administrator of delayed urgent issues

2. UML — Activity Diagram

1. Student identifies facility problem
↓
2. Submit photo + description + building/floor/room
↓
3. Validate and store report
↓
4. Multimodal AI/ML issue classification
↓
5. Duplicate-ticket similarity check
↓
6. Computer-vision hazard + severity/safety assessment
↓
7. ML priority prediction
↓
8. Automatic maintenance-department routing
↓
9. Maintenance staff accepts task
↓
10. Status: In Progress → Resolved
↓
11. Student receives status notification
↓
12. Response-time monitor checks target
↓
If delayed: reminder → administrator notification → escalation
↓
13. Administrator analytics + recurring-problem monitoring

3. UML — Sequence Diagram

Student	Frontend	Backend	AI/ML Engine	Database	Maintenance/Admin
Submit report	POST report	Validate + create		Store report	
		Send image/text	Classify; duplicate; hazard/severity; priority; routing	Store AI result	
	Show result	Create task		Store assignment	Receive task
		Notify staff		Store notification	Accept/update
	Show status	Process update		Store history	Resolve
		Check response target		Read task history	
Receive notification		Notify / escalate if delayed		Store notification	Admin notified
		Analytics query		Return aggregates	View dashboard

4. Mockup / Wireframe

These are low-fidelity layouts for the planned application flow.

A. Student Dashboard

+-----+
| CAMPUS MAINTENANCE [Bell] [User] |
+-----+
| Welcome, Student |
| |
| [+ REPORT A PROBLEM] |
| |
| MY REPORTS |
| [Broken AC | Room 301 | IN PROGRESS] |
| [Broken Light | Room 204 | RESOLVED] |
+-----+

B. Report Issue

+-----+
| REPORT CAMPUS ISSUE |
+-----+
| Photo: [UPLOAD / CAMERA] |
| Building: [Engineering Building v] |
| Floor: [3 v] Room: [301] |
| Description: |
| [Water is leaking from the AC...]|
| |
| [SUBMIT REPORT] |
+-----+

C. AI Analysis Result

+-----+
| AI ANALYSIS |
+-----+
| Category: HVAC / Air Conditioning |
| Confidence: 94% |
| Hazard: Water Leakage |
| Safety/Severity: 7.8 / 10 |
| Duplicate: No matching active incident |
| Priority: HIGH |
| Department: HVAC Maintenance |
+-----+

D. Maintenance Staff Dashboard

+-----+
| MAINTENANCE PORTAL |
+-----+
| CRITICAL – Exposed Electrical Wiring |
| Engineering Building / Room 301 |
| Safety Risk: HIGH |
| [ACCEPT] [VIEW REPORT] |
| |
| MEDIUM – Broken Classroom Light |
| Building B / Room 204 |
| [ACCEPT] [VIEW REPORT] |
+-----+

E. Administrator Dashboard

+-----+
| ADMIN DASHBOARD |
+-----+
| ACTIVE | CRITICAL | RESOLVED | ESCALATED |
| 27 | 3 | 142 | 3 |
+-----+
| Category / Department workload |
| Electrical | HVAC | Plumbing | IT |
| Recurring Problems: 7 |
+-----+

5. ER Diagram — Database Structure

The ER structure supports reporting, AI analysis, maintenance dispatch, status tracking, notifications, and feedback.

Entity	Primary Key	Key Fields / Relationships
USERS	user_id	name, student_id, email, role
REPORTS	report_id	user_id → USERS; location; description; image; category; status; created_at
AI_ANALYSIS	analysis_id	report_id → REPORTS; confidence; hazard; severity_score; duplicate_score; priority; department
DEPARTMENTS	department_id	name; description
MAINTENANCE_TASKS	task_id	report_id → REPORTS; department_id → DEPARTMENTS; staff_id → USERS; priority; status; timestamps
STATUS_HISTORY	history_id	report_id → REPORTS; changed_by → USERS; status; timestamp
NOTIFICATIONS	notification_id	user_id → USERS; report_id → REPORTS; message; is_read; created_at
FEEDBACK	feedback_id	report_id → REPORTS; user_id → USERS; rating; comment; created_at

Relationship Map

USERS	1 ■■■ N	REPORTS
REPORTS	1 ■■■ 1	AI_ANALYSIS
REPORTS	1 ■■■ N	STATUS_HISTORY
REPORTS	1 ■■■ 0..1	MAINTENANCE_TASKS
DEPARTMENTS	1 ■■■ N	MAINTENANCE_TASKS
USERS	1 ■■■ N	MAINTENANCE_TASKS (staff)
USERS	1 ■■■ N	NOTIFICATIONS
REPORTS	1 ■■■ N	NOTIFICATIONS
REPORTS	1 ■■■ 0..1	FEEDBACK

6. Architecture Summary

Layer	Planned Technology / Responsibility
Frontend	React.js — student, maintenance staff, administrator interfaces
Backend	Python + FastAPI — REST API, authentication, reporting workflow, routing, escalation
Database	PostgreSQL — users, reports, departments, tasks, history, notifications, feedback
AI/ML	Multimodal classification, duplicate detection, computer vision hazard/severity assessment, ML priority prediction
Testing	API/system testing, usability testing, and model evaluation

Scope note: No QR-based equipment identification is required. Reports use photo, text, and campus location information.

Implementation order: Requirements → database → backend/report workflow → React interfaces → AI baseline → classification/duplicate detection → hazard/severity → priority/routing → dispatch/escalation → analytics → integration/testing.